



Universidad Politécnica de Querétaro

Ingeniería en Sistemas computacionales

Materia: Programación web

“Documento de investigación – HU_1.6”

Docente: José Javier Moya Moya

Por: Andrés Joshua León Barranco, Alberto Luna Rufino, Eduardo Loredo García, Jorge Armando López Morales, Diego Jiménez Vargas y Saúl Alejandro Maldonado Ladrillero

Grupo: S-204

10/nov/2025

1. Objetivo

El objetivo de esta auditoría es revisar todos los archivos Seeder existentes en database/seeder/ para identificar por qué el comando `php artisan db:seed` puede estar fallando o arrojando errores. Se documentarán los problemas encontrados y se establecerá un plan de corrección para el Sprint 2.

2. Archivos auditados

- DatabaseSeeder.php
- UserSeeder.php
- RestaurantDetailSeeder.php
- PlatilloSeeder.php
- FaqSeeder.php

3. Análisis y errores identificados

Hallazgo 1: DatabaseSeeder.php (Correcto)

El archivo principal de seeders llama a los demás en el orden correcto. UserSeeder se ejecuta primero, lo cual es esencial.

Hallazgo 2: UserSeeder.php y FaqSeeder.php (Correctos)

Estos dos seeders no tienen dependencias externas y solo crean datos. Su lógica es correcta.

Hallazgo 3: RestaurantDetailSeeder.php (Correcto)

Este seeder implementa la manera correcta de manejar las relaciones. Primero busca al usuario (`$restaurante1 = User::where(...)`) y luego comprueba que exista (`if ($restaurante1) { ... }`) antes de intentar usar su ID. Este es el patrón a seguir.

Hallazgo 4: PlatilloSeeder.php (¡ERROR CRÍTICO!)

Aquí se ha identificado el error principal que probablemente causa que los seeders fallen.

- **El Problema:** El archivo PlatilloSeeder.php intenta obtener el ID del restaurante (\$restauranteItaliano->id) *dentro* de la definición del array \$platillos.
- **El Fallo:** Si UserSeeder falla o el email pizza@bella.com no se encuentra, \$restauranteItaliano será null. El código fallará con un error fatal (Attempt to read property "id" on null) en la línea ['user_id' => \$restauranteItaliano->id, ...].
- **Comparación:** Este seeder no sigue el patrón de seguridad que sí implementa RestaurantDetailSeeder.php.

4. Plan de corrección (Tareas para Sprint 2)

Para las tareas HU_2.2, HU_2.3 y HU_2.4, se debe refactorizar el PlatilloSeeder.php para que sea robusto y maneje la posibilidad de que un usuario no exista.

Plan para PlatilloSeeder.php

El PlatilloSeeder.php debe reescribirse para seguir el mismo patrón que RestaurantDetailSeeder.php.

Lógica de corrección:

1. Buscar los usuarios restaurantes al inicio (como ya lo hace).
2. Definir los arrays de platillos *sin* el user_id, pero agrupados por restaurante.
3. Usar un foreach para cada grupo de platillos, *dentro* de un if que compruebe que el restaurante existe.

Ejemplo de código corregido (borrador):

```
// ... (imports)
```

```
public function run(): void
```

```

{

// 1. Buscar los restaurantes

$restauranteItaliano = User::where('email', 'pizza@bella.com')->first();

$restauranteAmericano = User::where('email', 'burger@queen.com')->first();

// ... (etc.)


// 2. Definir platillos SIN user_id

$platillosItalianos = [

    ['nombre' => 'Pizza Margherita', 'descripcion' => '...', 'precio' => 120, ...],

    ['nombre' => 'Lasaña Boloñesa', 'descripcion' => '...', 'precio' => 150, ...],

];


$platillosAmericanos = [

    ['nombre' => 'Classic Burger', 'descripcion' => '...', 'precio' => 130, ...],

];


// 3. Insertar platillos DE FORMA SEGURA

if ($restauranteItaliano) {

    foreach ($platillosItalianos as $platillo) {

        // Añadir el user_id aquí

        $platillo['user_id'] = $restauranteItaliano->id;

        Platillo::create($platillo);

    }

}

```

```
}
```

```
if ($restauranteAmericano) {
```

```
    foreach ($platillosAmericanos as $platillo) {
```

```
        $platillo['user_id'] = $restauranteAmericano->id;
```

```
        Platillo::create($platillo);
```

```
    }
```

```
}
```

```
// ... (etc.)
```

```
}
```